



Faculty of Computer and
Data science Alexandria
University
2024 - 2025

SUBJECT

Distributed Processing

TITLE

Final project

By:

Loubid Emad Mohamed	23012047
Marwan Mohamed Zein	23011523
Huda Ali Fouad Mohamed	23011605
Habiba Hamdy Ali	23011064
Habiba Yehia Shaaban	23011066
Rahma Mohamed Abdo	23010107

Supervision:

Prof. Dr. Hatem Abdelkader

Eng. Sara Mohamed

Table of Contents

1. Introduction.....	1
2. Containerization with Docker.....	3
3. Data Storage in HDFS.....	4
4. Data Processing with MapReduce and Hive	5
5. Data Visualization and Analysis.....	9

1. Introduction

The main goal of this project is to collect weather-related datasets from the internet and process them to extract meaningful insights. The project workflow is designed to ensure scalability, reproducibility, and efficient data processing across multiple environments. The main steps are as follows:

1-Data Collection and Preprocessing

- Weather datasets are collected from online sources.
- The data is cleaned and preprocessed to handle missing values, inconsistencies, and formatting issues, ensuring it is ready for further processing.

2- Containerization with Docker

- A Docker file is created to package the project environment, including all necessary dependencies.
- Using Docker allows the project to run consistently on multiple machines without installation conflicts or environmental issues.

3- Data Storage in HDFS

- The cleaned data is uploaded to the Hadoop Distributed File System (HDFS) for scalable storage and efficient distributed processing.

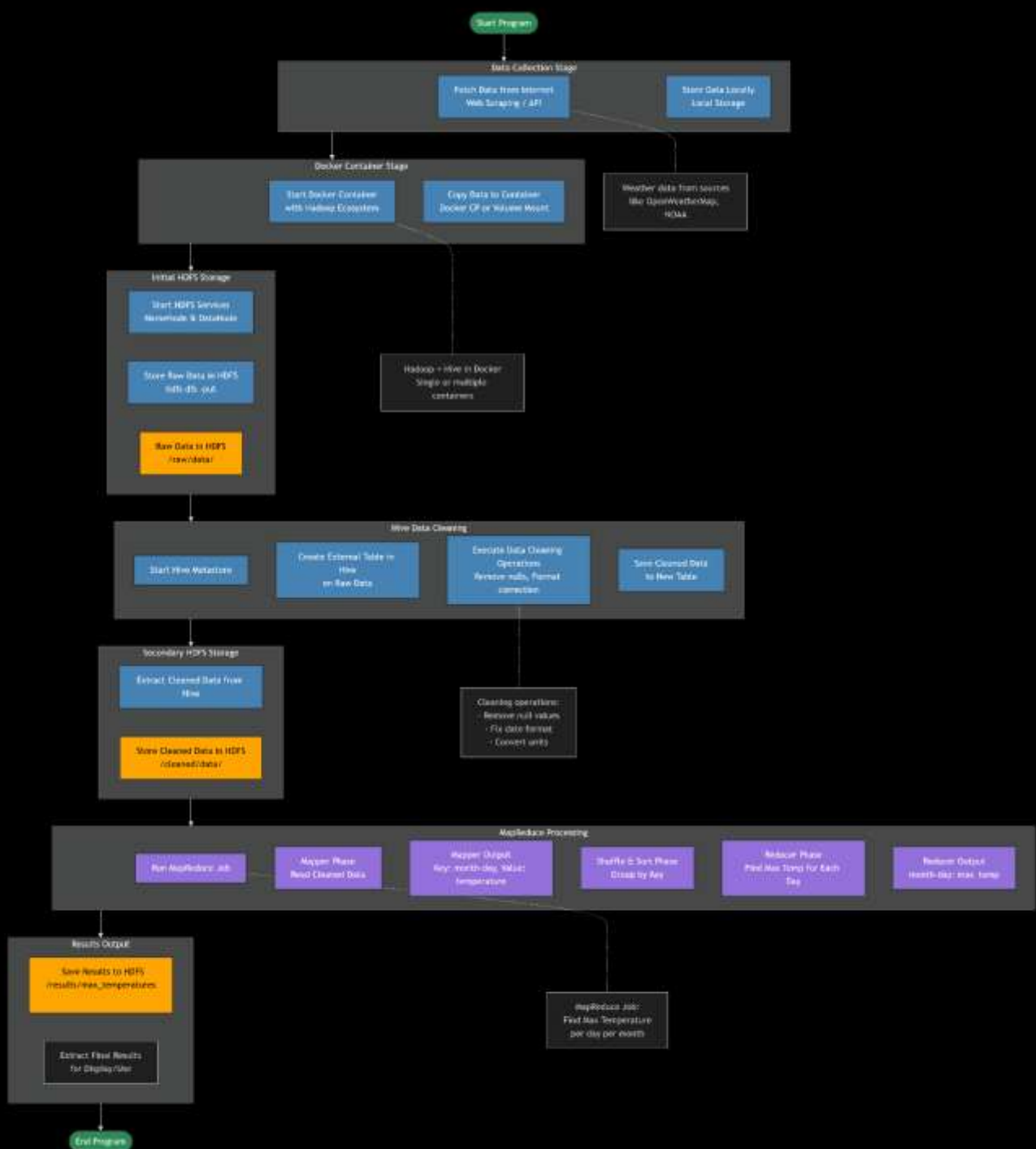
4- Data Processing with MapReduce and Hive

- MapReduce jobs are applied to perform distributed computations on the dataset.
- Hive is used to query and organize the data for further analysis, providing a structured interface on top of HDFS.

5- Data Visualization and Analysis

- Finally, the processed data is analyzed and visualized to extract meaningful patterns and insights.
- Various visualization techniques are applied to interpret weather trends and present the results clearly.

This workflow ensures that the entire process, from raw data collection to final analysis, is **scalable, reproducible, and efficient**, leveraging big modern data tools and containerization technology.



2. Containerization with Docker

Below is a clean, professional, university-level paragraph based on everything you told me earlier:

To ensure a consistent and reproducible environment for running Big Data tools, we used **Docker** to containerize the entire setup. First, we created a **Docker file** that defines all the components required inside the container, including the operating system (Ubuntu), Hadoop, Hive, and the necessary configurations for HDFS and MapReduce. After preparing the Docker file and the related configuration files, the whole project structure was uploaded to **GitHub** so it can be easily cloned on any machine. Once Docker was installed locally, we built a custom Docker **image** from the Docker file, which encapsulates the Big Data environment with all pre-configured dependencies. Based on this image, we created and ran a **container**, which acts as an isolated workspace where we can upload the dataset and execute Big Data processing tasks. Inside the container, we are able to interact with HDFS, run MapReduce jobs, execute Hive queries, and perform the full ETL workflow without affecting the host system. This approach guarantees consistency, portability, and easier deployment across different machines.

3. Data Storage in HDFS

- First make directory in hdfs to store data in it

```
hadoop@f78b85de0265:~$ hdfs dfs -mkdir /user/hadoop/weather-project
```

- Second, we upload our two data files into the HDFS directory.

```
hadoop@f78b85de0265:~$ hdfs dfs -put file1.csv /user/hadoop/weather-project/
```

```
hadoop@f78b85de0265:~$ hdfs dfs -put file2.csv /user/hadoop/weather-project/
```

- After uploading the files, I ran the [hdfs dfs -ls](#) command to verify that the files were successfully placed in the specified directory.

```
hadoop@f78b85de0265:~$ hdfs dfs -ls /user/hadoop/weather-project
2025-12-05 17:24:27,041 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform.
Found 3 items
drwxr-xr-x   - hadoop supergroup          0 2025-12-05 16:22 /user/hadoop/weather-project/cleaned_weather
-rw-r--r--   1 hadoop supergroup    251904 2025-12-03 19:12 /user/hadoop/weather-project/file1.csv
-rw-r--r--   1 hadoop supergroup    268382 2025-12-03 19:12 /user/hadoop/weather-project/file2.csv
hadoop@f78b85de0265:~$
```

- Next, I created another directory in HDFS to store the cleaned version of the dataset. After performing the data cleaning using Hive, I saved the processed data into this new directory so that it can be used later for MapReduce processing

```
hadoop@f78b85de0265:~$ hdfs dfs -mkdir /user/hadoop/weather-project/cleaned_weather
```

4. Data Processing with MapReduce and Hive

4.1 Hive:

- running hiveserver2 starts the Hive server that allows clients to connect and access the Hive database.

```
hadoop@f78b85de0265:/opt$ hiveserver2
SLF4J: Class path contains multiple SLF4J bindings.
SLF4J: Found binding in [jar:file:/opt/hive/lib/log4j-slf4j-impl-2.18
SLF4J: Found binding in [jar:file:/opt/hadoop/share/hadoop/common/lib
SLF4J: See http://www.slf4j.org/codes.html#multiple_bindings for an e
SLF4J: Actual binding is of type [org.apache.logging.slf4j.Log4jLogge
2025-12-05 15:57:35: Starting HiveServer2
SLF4J: Class path contains multiple SLF4J bindings.
SLF4J: Found binding in [jar:file:/opt/hive/lib/log4j-slf4j-impl-2.18
SLF4J: Found binding in [jar:file:/opt/hadoop/share/hadoop/common/lib
SLF4J: See http://www.slf4j.org/codes.html#multiple_bindings for an e
SLF4J: Actual binding is of type [org.apache.logging.slf4j.Log4jLogge
Hive Session ID = 9d98c7ec-50cf-4fdb-b219-d60863c95622
Fri Dec 05 15:57:52 GMT 2025 Thread[main,5,main] java.io.FileNotFound
Fri Dec 05 15:57:53 GMT 2025 Thread[main,5,main] Ignored duplicate pr
/apache/derby/modules.properties
Fri Dec 05 15:57:53 GMT 2025 Thread[main,5,main] Ignored duplicate pr
/org/apache/derby/modules.properties
Fri Dec 05 15:57:53 GMT 2025 Thread[main,5,main] Ignored duplicate pr
/org/apache/derby/modules.properties
Fri Dec 05 15:57:53 GMT 2025 Thread[main,5,main] Ignored duplicate pr
```

- Here we start write Queries for make tables can clean our data

```
hadoop@f78b85de0265:~$ beeline -u jdbc:hive2://localhost:10000 -n hadoop
SLF4J: Class path contains multiple SLF4J bindings.
SLF4J: Found binding in [jar:file:/opt/hive/lib/log4j-slf4j-impl-2.18.0.jar!
SLF4J: Found binding in [jar:file:/opt/hadoop/share/hadoop/common/lib/slf4j-
SLF4J: See http://www.slf4j.org/codes.html#multiple_bindings for an explanat
SLF4J: Actual binding is of type [org.apache.logging.slf4j.Log4jLoggerFactor
Connecting to jdbc:hive2://localhost:10000
Connected to: Apache Hive (version 4.0.1)
Driver: Hive JDBC (version 4.0.1)
Transaction isolation: TRANSACTION_REPEATABLE_READ
Beeline version 4.0.1 by Apache Hive
0: jdbc:hive2://localhost:10000>
```

- Create tables for our csv files

```
CREATE EXTERNAL TABLE raw_weather (
  STATION          BIGINT,
  DATE             STRING,
  SOURCE           INT,
  LATITUDE         DOUBLE,
  LONGITUDE        DOUBLE,
  ELEVATION        DOUBLE,
  NAME             STRING,
  REPORT_TYPE      STRING,
  CALL_SIGN        STRING,
  QUALITY_CONTROL  STRING,
  WND              STRING,
  CIO              STRING,
  VIS              STRING,
  TMP              STRING,
  DEN              STRING,
  SLP              STRING,
  ALI              STRING
)
ROW FORMAT SERDE 'org.apache.hadoop.hive.serde2.OpenCSVSerde'
WITH SERDEPROPERTIES (
  'separatorChar' = ',',
  'quoteChar' = '"'
)
STORED AS TEXTFILE
LOCATION '/user/hadoop/weather-project/cleaned_weather/';
```


- The SQL query creates a new table called `clean_weather` by extracting and cleaning weather data from the `raw_weather` table. It processes date fields, extracts year, and converts temperature, dew point, and pressure values from raw string formats into usable numeric values, handling missing or invalid
- **Note:** we use Another way to clean our data to check by python script:

```
CREATE TABLE clean_weather AS
SELECT
    station,
    name,
    latitude,
    longitude,
    elevation,

    -- clean date fields
    date AS raw_date,
    substr(date, 1, 10) AS date_day,
    cast(substr(date, 1, 4) as int) AS year,

    -- temperature
    CASE
        WHEN split(tmp, ',')[0] NOT IN ('+9999', '-9999')
        THEN cast(split(tmp, ',')[0] AS int) / 10.0
        ELSE NULL
    END AS temperature,

    -- dew point
    CASE
        WHEN split(dew, ',')[0] NOT IN ('+9999', '-9999')
        THEN cast(split(dew, ',')[0] AS int) / 10.0
        ELSE NULL
    END AS dew_point,

    -- pressure
    CASE
        WHEN split(elp, ',')[0] NOT IN ('+99999', '-99999')
        THEN cast(split(elp, ',')[0] AS int) / 10.0
        ELSE NULL
    END AS pressure

FROM raw_weather;
```

```
import pandas as pd

# ----- CONFIG -----
FILE1 = "file1.csv"
FILE2 = "file2.csv"
OUTPUT = "cleaned_weather.csv"
# -----

def extract_temperature(tmp_value):
    """
    Extract temperature from TMP field.
    TMP format from NOAA ISD: "000000,1,N,9"
    First value = temperature in tenths of °C.
    Missing temperature = '+9999'
    """
    if pd.isna(tmp_value):
        return None

    parts = str(tmp_value).split(',')
    if len(parts) == 0:
        return None

    raw = parts[0] # First field should be temperature

    # Remove non-digit chars
    raw = raw.replace("+", "").replace("-", "").strip()

    if raw in ["9999", "99999", ""]:
        return None

    # Convert tenths of °C to standard °C
    try:
        return float(raw) / 10.0
    except:
        return None
```

```
def extract_temperature(row):
    """
    # Convert tenths of °C to standard °C
    try:
        return float(row) / 10.0
    except:
        return None

def main():
    print("Loading CSV files...")
    df1 = pd.read_csv(FILE1)
    df2 = pd.read_csv(FILE2)

    print("Merging...")
    df = pd.concat([df1, df2], ignore_index=True)

    print("Cleaning DATE column...")
    df["DATE"] = pd.to_datetime(df["DATE"], errors="coerce")

    print("Extracting clean TMP values...")
    df["CLEAN_TMP"] = df["TMP"].apply(extract_temperature)

    print("Extracting clean DEW values...")
    df["CLEAN_DEW"] = df["DEW"].apply(extract_temperature)

    print("Removing rows where DATE or CLEAN_TMP are invalid...")
    df = df.dropna(subset=["DATE", "CLEAN_TMP"])

    print("Saving cleaned CSV...")
    df.to_csv(OUTPUT, index=False)

    print("\n✓ DONE: cleaned file saved as:", OUTPUT)

if __name__ == "__main__":
    main()
```


4.2. MapReduce:

Mapper Code:

```
1 import java.io.IOException;
2 import java.util.ArrayList;
3 import java.util.HashMap;
4 import org.apache.hadoop.io.DoubleWritable;
5 import org.apache.hadoop.io.LongWritable;
6 import org.apache.hadoop.io.Text;
7 import org.apache.hadoop.mapreduce.Mapper;
8
9 public class MonthTempMapper extends Mapper<LongWritable, Text, Text, DoubleWritable> {
10
11     private final static DoubleWritable tempWritable = new DoubleWritable();
12     private Text monthKey = new Text();
13
14     // Simple CSV parser
15     private ArrayList<String> parseCSV(String line) {
16         ArrayList<String> cols = new ArrayList<>();
17         StringBuilder cur = new StringBuilder();
18         boolean inQuotes = false;
19         for (int i = 0; i < line.length(); i++) {
20             char c = line.charAt(i);
21             if (c == '"') {
22                 inQuotes = !inQuotes;
23             } else if (c == ',' && !inQuotes) {
24                 cols.add(cur.toString());
25                 cur.setLength(0);
26             } else {
27                 cur.append(c);
28             }
29         }
30         cols.add(cur.toString());
31         return cols;
32     }
33
34     @Override
35     protected void map(LongWritable key, Text value, Context context)
36         throws IOException, InterruptedException {
37
38         String line = value.toString().trim();
39         if (line.isEmpty()) return;
40
41         String low = line.toLowerCase();
42         if (low.startsWith("station,") || low.contains("station,date")) return;
43
44         ArrayList<String> cols = parseCSV(line);
45
46         int idxDate = 1; // adjust if your date column is different
47         int idxTmp = 13; // adjust to the temperature column
48
49         if (cols.size() <= idxTmp) return;
50
51         String dateStr = cols.get(idxDate).trim(); // e.g., 1918-12-31 00:00:00
52         String tmpStr = cols.get(idxTmp).trim();
53
54         // Skip invalid values
55         if (tmpStr.isEmpty() || tmpStr.equals("9999") || tmpStr.equals("99999") || tmpStr.equals("999.9")) {
56             return;
57         }
58
59         double temp;
60         try {
61             temp = Double.parseDouble(tmpStr.replaceAll("[^0-9\\.\\-]", "")) / 10.0; // divide by 10 if tenths
62         } catch (NumberFormatException e) {
63             return;
64         }
65
66         // sanity check
67         if (temp < -50 || temp > 60) return;
68
69         // extract month: YYYY-MM
70         String month = null;
71         if (dateStr.length() >= 7) {
72             month = dateStr.substring(0, 7); // "1918-12"
73         }
74         if (month == null) return;
75
76         monthKey.set(month);
77         tempWritable.set(temp);
78         context.write(monthKey, tempWritable);
79     }
80 }
```

Reducer Code:

```
import java.io.IOException;
import org.apache.hadoop.io.DoubleWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Reducer;

public class MonthTempReducer extends Reducer<Text, DoubleWritable, Text, DoubleWritable> {

    @Override
    protected void reduce(Text key, Iterable<DoubleWritable> values, Context context)
        throws IOException, InterruptedException {

        double monthMax = -Double.MAX_VALUE;

        for (DoubleWritable v : values) {
            double t = v.get();
            if (t > monthMax) monthMax = t;
        }

        context.write(key, new DoubleWritable(monthMax));
    }
}
```

Driver Code:

```
1  import org.apache.hadoop.conf.Configuration;
2  import org.apache.hadoop.fs.Path;
3  import org.apache.hadoop.io.DoubleWritable;
4  import org.apache.hadoop.io.Text;
5  import org.apache.hadoop.mapreduce.Job;
6  import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
7  import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
8
9  public class MonthTempDriver {
10
11      public static void main(String[] args) throws Exception {
12          if (args.length != 2) {
13              System.err.println("Usage: MonthTempDriver <input path> <output path>");
14              System.exit(-1);
15          }
16
17          Configuration conf = new Configuration();
18          Job job = Job.getInstance(conf, "Monthly Max Temperature");
19
20          job.setJarByClass(MonthTempDriver.class);
21          job.setMapperClass(MonthTempMapper.class);
22          job.setReducerClass(MonthTempReducer.class);
23
24          job.setMapOutputKeyClass(Text.class);
25          job.setMapOutputValueClass(DoubleWritable.class);
26
27          job.setOutputKeyClass(Text.class);
28          job.setOutputValueClass(DoubleWritable.class);
29
30          FileInputFormat.addInputPath(job, new Path(args[0]));
31          FileOutputFormat.setOutputPath(job, new Path(args[1]));
32
33          System.exit(job.waitForCompletion(true) ? 0 : 1);
34      }
35  }
```

- compile java files and produce JAR file

```
hadoop@f78b85de0265:~/src$ javac -classpath $(hadoop classpath) -d classes MonthTempMapper.java MonthTempReducer.java MonthTempDriver.java
hadoop@f78b85de0265:~/src$ jar -cvf MonthTempJob.jar -C classes/ .
added manifest
adding: MonthTempDriver.class(in = 1643) (out= 909)(deflated 44%)
adding: MonthTempMapper.class(in = 3289) (out= 1611)(deflated 51%)
adding: MonthTempReducer.class(in = 1664) (out= 706)(deflated 57%)
```

- Now we use JAR file to start MapReduce job in Hadoop

```
hadoop@f78b85de0265:~/src$ hadoop jar MonthTempJob.jar MonthTempDriver /user/hadoop/weather-project/cleaned_weather /output/month_temp2
2025-12-05 18:15:19,342 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
2025-12-05 18:15:22,170 INFO client.DefaultHARMPFailoverProxyProvider: Connecting to ResourceManager at localhost/127.0.0.1:8032
2025-12-05 18:15:23,667 WARN mapreduce.JobResourceUploader: Hadoop command-line option parsing not performed. Implement the Tool interface and execute your application with ToolRunner to remedy this.
2025-12-05 18:15:23,755 INFO mapreduce.JobResourceUploader: Disabling Erasure Coding for path: /tmp/hadoop-yarn/staging/hadoop/.staging/job_1764955056481_0004
2025-12-05 18:15:25,181 INFO input.FileInputFormat: Total input files to process : 1
2025-12-05 18:15:25,431 INFO mapreduce.JobSubmitter: number of splits:1
2025-12-05 18:15:26,197 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1764955056481_0004
2025-12-05 18:15:26,199 INFO mapreduce.JobSubmitter: Executing with tokens: []
2025-12-05 18:15:27,057 INFO conf.Configuration: resource-types.xml not found
2025-12-05 18:15:27,058 INFO resource.ResourceUtils: Unable to find 'resource-types.xml'.
2025-12-05 18:15:27,330 INFO impl.YarnClientImpl: Submitted application application_1764955056481_0004
2025-12-05 18:15:27,574 INFO mapreduce.Job: The url to track the job: http://f78b85de0265:8088/proxy/application_1764955056481_0004/
2025-12-05 18:15:27,577 INFO mapreduce.Job: Running job: job_1764955056481_0004
2025-12-05 18:15:48,139 INFO mapreduce.Job: Job job_1764955056481_0004 running in uber mode : false
2025-12-05 18:15:48,149 INFO mapreduce.Job: map 0% reduce 0%
2025-12-05 18:16:00,660 INFO mapreduce.Job: map 100% reduce 0%
2025-12-05 18:16:15,495 INFO mapreduce.Job: map 100% reduce 100%
```

The Final Output of MapReduce:

```
hadoop@f78b85de0265:~/src$ hdfs dfs -cat /output/month_temp/part-r-00000
1916-01 50.1
1916-02 50.1
1916-03 56.1
1916-04 56.1
1916-05 56.1
1916-06 56.1
1916-07 50.1
1916-08 50.1
1916-09 56.1
1916-10 50.1
1916-11 56.1
1916-12 28.1
hadoop@f78b85de0265:~/src$
```

This output explains which is highest temperature in each month

5. Data Visualization and Analysis

